SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

Sl. No	Questions	Bloom's Level
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

Sl. No	Questions	Bloom's Level
9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

*I **Mugesh S (192411055)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

ANSWERS

Question 1: ATM Withdrawal Module

1. Problem Analysis

The ATM withdrawal module is partially tested, and only input-output behaviour is available. The system validates withdrawal limits, account balance, and error messages. Reverse engineering helps identify the original testing approach and improve missing test coverage.

2. Functional Decomposition

The ATM withdrawal system can be divided into the following components:

1. Input Validation: Checks whether the withdrawal amount is numeric and positive.
2. Withdrawal Limit Validation: Ensures the amount is within the permitted transaction limits.
3. Balance Validation: Checks whether the account has sufficient funds.
4. Transaction Processing: Deducts the approved amount from the account.
5. Error Handling: Displays appropriate messages for invalid transactions.
6. Receipt Generation: Displays or prints the transaction result.

3. Equivalence Class Partitioning (ECP)

Assume the permitted withdrawal amount is ₹100 to ₹25,000 per transaction. These limits are illustrative.

Test Case	Input Amount	Equivalence Class	Expected Result
TC1	₹0	Zero amount	Reject
TC2	-₹100	Negative amount	Reject
TC3	₹50	Below minimum	Reject
TC4	₹500	Valid amount	Accept if balance is sufficient
TC5	₹25,000	Maximum valid amount	Accept if balance is sufficient
TC6	₹25,001	Above maximum	Reject
TC7	Alphabetic input	Invalid format	Display validation error

4. Boundary Value Analysis (BVA)

Test values around the assumed withdrawal limits:

- Minimum boundary: ₹99, ₹100, ₹101.
- Maximum boundary: ₹24,999, ₹25,000, ₹25,001.

Additional tests should check account balance boundaries, including a withdrawal equal to the available balance and one exceeding it by ₹1.

5. Evaluation of Existing Testing

The observed input-output behaviour suggests that black-box techniques such as equivalence partitioning and boundary value analysis may have been used. However, this cannot be confirmed without the original test cases.

Possible gaps include insufficient balance, daily withdrawal limits, invalid PINs, network failures, duplicate requests, and transaction rollback.

6. Improved Test Strategy

- Apply ECP to valid and invalid amounts.
- Apply BVA around transaction and balance limits.
- Test valid and invalid PINs.
- Verify that failed withdrawals do not reduce the balance.
- Test network failure during transaction processing.
- Verify that duplicate requests do not cause duplicate debits.
- Perform regression testing after correcting defects.

Conclusion: ECP reduces redundant tests, while BVA targets errors near critical limits. Combining them with transaction-integrity and error-handling tests improves ATM reliability.

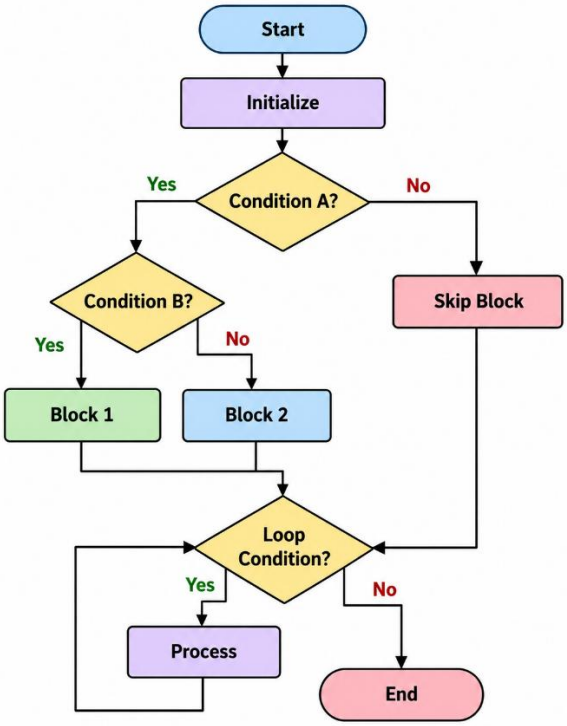
Question 2: Software Module — White-Box Testing and Path Coverage

1. Problem Analysis

The module provides execution traces and code snippets but has no test documentation. The objective is to reconstruct the control flow, identify decision points and loops, evaluate coverage, and design a path-testing strategy.

Because the actual code snippets and execution traces are not included in the assessment, the following is an illustrative reconstruction.

2. Conceptual Control Flow Graph (CFG)



Explanation:

- Condition A determines whether the main processing block executes.
- Condition B selects between Block 1 and Block 2.
- The loop condition determines whether processing repeats.
- The loop terminates when its condition becomes false.

3. Decision Points and Loops

Component	Purpose	Testing Requirement
Condition A	Controls the main block	Test True and False
Condition B	Selects between two blocks	Test both outcomes
Loop condition	Controls repetition	Test entry, repetition, and exit
Block 1	Executes when B is True	Verify execution
Block 2	Executes when B is False	Verify execution
Skip Block	Executes when A is False	Verify the alternate path

4. Independent Execution Paths

The following paths should be tested:

1. Start → Initialize → A=False → Skip Block → Loop condition → End.
2. Start → Initialize → A=True → B=True → Block 1 → Loop condition → End.
3. Start → Initialize → A=True → B=False → Block 2 → Loop condition → End.
4. Start → Initialize → A=True → B=True → Block 1 → Loop condition → Process → Loop condition → End.
5. Start → Initialize → A=True → B=False → Block 2 → Loop condition → Process → Loop condition → End.

The last two paths represent repeated processing. They are illustrative paths, not a verified complete path set for the unavailable source code.

5. Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths.

$$V(G) = E - N + 2P$$

Where:

- E= number of edges.
- N= number of nodes.
- P = number of connected components.

For a single connected graph:

$$V(G) = D + 1$$

Here, D is the number of decision nodes.

For the simplified graph containing three binary decision points, the estimated cyclomatic complexity is:

$$V(G) = 3 + 1 = 4$$

Therefore, at least four independent paths should be selected for basis path testing of that graph.

6. Improved Testing Strategy

1. Construct the CFG from the actual source code.
2. Identify all decisions and loop conditions.

3. Calculate cyclomatic complexity.
4. Select a basis set of independent paths.
5. Test each decision outcome.
6. Test loops with zero, one, and multiple iterations.
7. Use statement, branch, and path-oriented coverage reports to identify missing tests.
8. Perform regression testing after fixing defects.

Conclusion: CFG-based basis path testing provides a systematic approach to white-box testing. It helps identify untested branches, loop errors, and defects in alternative execution paths.

Question 3: Web Application Login and Transaction System

1. Problem Analysis

The web application contains login and transaction functions. Only defect logs and test results are available, so the original testing techniques must be inferred from the observed failures.

For example, defects involving amounts near transaction limits may indicate inadequate boundary value analysis, while failures involving particular conditions may indicate insufficient branch or path coverage.

2. Functional Decomposition

The application can be divided into:

1. Login Validation: Validates username and password.
2. Authentication: Verifies user credentials.
3. Session Management: Maintains login state and session expiry.
4. Transaction Validation: Checks transaction amount and account information.
5. Transaction Processing: Performs the requested transaction.
6. Error Handling: Handles invalid credentials, insufficient balance, and processing failures.
7. Audit Logging: Records successful and failed transactions.

3. Likely Testing Techniques

Observed Defect Pattern	Possible Missing or Inadequate Technique
Errors near minimum and maximum amounts	Boundary Value Analysis

Invalid usernames or transaction formats accepted	Equivalence Class Partitioning
Failure in a specific decision branch	Branch Coverage
Incorrect behaviour after several conditions	Path Testing
Defects reappear after changes	Regression Testing

These are reasonable inferences, not confirmed descriptions of the original testing process.

4. Equivalence Class Partitioning

For login testing:

Input Class	**Example**	**Expected Result**
Valid credentials	Correct username and password	Login succeeds
Incorrect password	Valid username, wrong password	Login rejected
Unknown username	Unregistered username	Login rejected
Empty credentials	Blank username or password	Validation error
Malformed input	Invalid input format	Safely rejected

For transactions, use separate classes for valid amounts, zero amounts, negative amounts, amounts above the limit, and amounts exceeding the available balance.

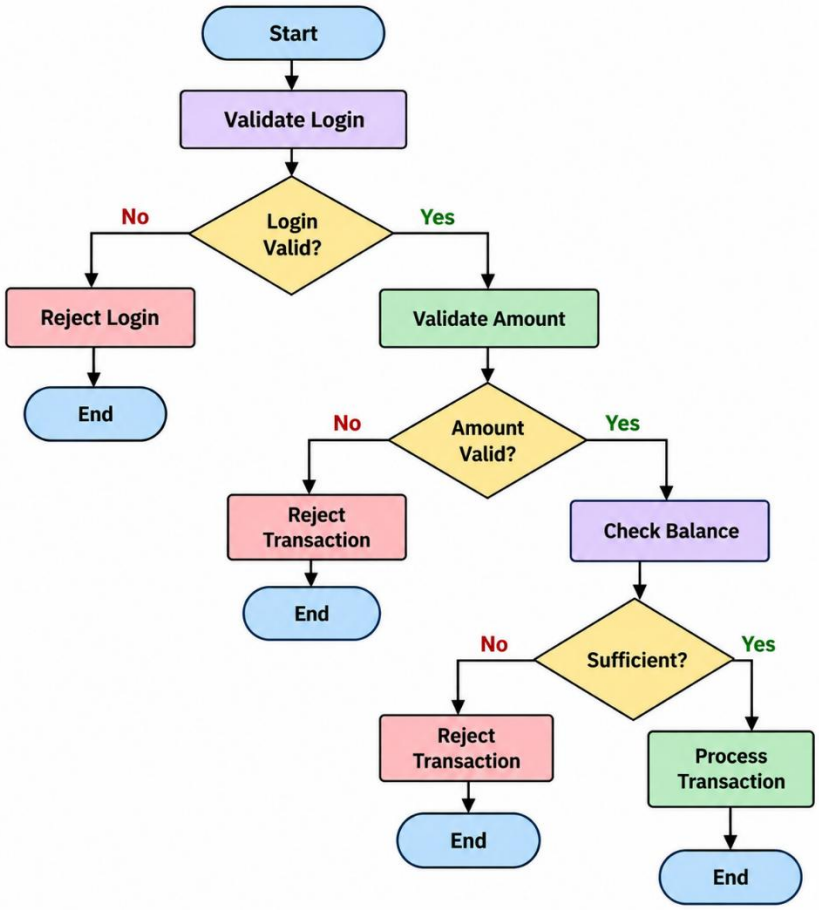
5. Boundary Value Analysis

Assume the transaction amount must be between ₹100 and ₹50,000.

Test Case	**Amount**	**Expected Result**
TC1	₹99	Reject
TC2	₹100	Accept if other conditions pass
TC3	₹101	Accept if other conditions pass
TC4	₹49,999	Accept if other conditions pass
TC5	₹50,000	Accept if other conditions pass
TC6	₹50,001	Reject

6. Path Testing

For a simplified transaction flow:



Tests must cover each True and False outcome, including failed authentication, invalid amounts, insufficient funds, and successful processing.

7. Improved Testing Strategy

- Apply ECP and BVA to login and transaction inputs.
- Use branch and basis path testing for internal logic.
- Test session expiry and repeated login attempts.
- Verify transaction rollback after failures.
- Test duplicate requests and concurrent transactions.
- Maintain regression tests for every defect found.

Conclusion: Combining black-box techniques with white-box path testing provides broader coverage of input validation, authentication decisions, and transaction processing.

Question 4: Legacy Inventory Management System

1. Problem Analysis

The inventory management system provides successful and failed transaction logs for stock updates and low-stock alerts. These logs can be analysed to infer stock validation rules, inventory boundaries, and alert conditions.

Since actual logs and business rules are not provided, the following limits are illustrative.

2. Functional Decomposition

The stock-update process consists of:

1. Product Identification: Verifies the product ID.
2. Quantity Validation: Checks whether the requested quantity is valid.
3. Stock Update: Increases or decreases the available quantity.
4. Stock-Level Validation: Prevents inventory from becoming negative.
5. Low-Stock Detection: Compares the remaining quantity against a threshold.
6. Alert Generation: Notifies the responsible user when stock is low.
7. Transaction Logging: Records successful and failed updates.

3. Equivalence Class Partitioning

Assume the minimum valid stock quantity is zero and the low-stock threshold is 10 units.

Test Case	**Stock Quantity**	**Class**	**Expected Result**
TC1	-1	Invalid negative stock	Reject update
TC2	0	Zero stock	Accept if zero inventory is permitted; trigger stock alert
TC3	5	Below threshold	Generate low-stock alert
TC4	10	At threshold	Apply the defined threshold rule
TC5	11	Above threshold	No low-stock alert
TC6	100	Normal stock	Update successfully
TC7	Non-numeric	Invalid input	Reject update

4. Boundary Value Analysis

Test values around the low-stock threshold:

- 9 units: below threshold.
- 10 units: exactly at threshold.
- 11 units: above threshold.

Also test stock updates that result in quantities of -1, 0, and 1. This identifies errors involving negative inventory and zero-stock handling.

5. Reconstructed Black-Box Test Design

Test Case	Initial Stock	Operation	Expected Result
TC1	20	Remove 5 units	Stock becomes 15
TC2	10	Remove 10 units	Stock becomes 0
TC3	5	Remove 6 units	Reject insufficient stock
TC4	9	Remove 1 unit	Stock becomes 8; low-stock alert remains applicable
TC5	10	Add 5 units	Stock becomes 15
TC6	0	Remove 1 unit	Reject update
TC7	5	Add 10 units	Stock becomes 15; verify alert clearing if specified

6. Improved Testing Strategy

1. Analyse transaction logs to identify repeated failure patterns.
2. Define valid and invalid stock quantities.
3. Apply BVA around zero stock and the alert threshold.
4. Test stock addition, stock removal, and invalid product IDs.
5. Verify that failed transactions do not change inventory.
6. Test concurrent stock updates to prevent lost updates.
7. Verify alert generation and clearing.
8. Perform regression testing.

Conclusion: ECP and BVA help reconstruct the hidden inventory rules from observed behaviour. Transaction-integrity and alert tests ensure that stock levels remain correct and low-stock notifications operate reliably.

Question 5: Library Management Module — Resource-Intensive Paths

1. Problem Analysis

The library management module is available only as a compiled executable file, together with CPU and memory execution profiles collected during stress testing. Execution spikes may indicate resource-intensive loops, repeated database operations, inefficient searches, or memory-intensive processing.

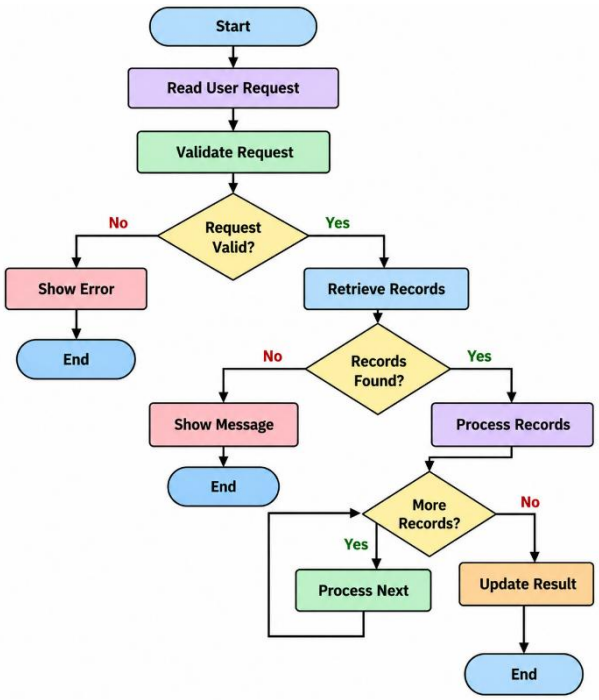
A compiled executable and performance profile do not reveal the exact source-level control flow. Therefore, the CFG below is conceptual and must be validated against available execution evidence.

2. Functional Decomposition

The module may contain the following components:

1. Input Processing: Accepts search terms or library operations.
2. Record Retrieval: Reads book and member records.
3. Availability Validation: Checks whether a book is available.
4. Transaction Processing: Handles borrowing and returning.
5. Record Update: Updates the database.
6. Result Generation: Displays transaction results.
7. Error Handling: Handles missing records and failed operations.

3. Conceptual Control Flow Graph (CFG)



The record-processing loop may become resource-intensive when the dataset is large or the termination condition is incorrect.

4. Mapping Resource Spikes to Possible Causes

Observed Profile	Possible Cause	Testing Focus
High CPU usage	Repeated processing loop	Loop termination and large datasets
Increasing memory usage	Memory leak or excessive allocation	Long-running stress tests
CPU spike during search	Inefficient record scanning	Large record collections
Delay during updates	Repeated database operations	Transaction and query performance
Crash under heavy load	Resource exhaustion	Capacity and recovery testing

These are hypotheses to investigate, not confirmed causes.

5. Reconstructed Path Testing Strategy

- Test invalid and valid requests.
- Test empty, small, and large record collections.
- Test records found and records not found.
- Exercise loop entry, repeated execution, and termination.
- Test borrowing and returning with valid and invalid records.
- Monitor CPU usage, memory consumption, response time, and errors.
- Repeat operations over extended periods to detect memory leaks.
- Test recovery after database or resource failures.

6. Impact on System Stability

Resource-intensive paths can cause slow responses, excessive memory consumption, and crashes. Testing these paths under normal, peak, and sustained workloads helps identify resource limits and verify that the system recovers gracefully.

Conclusion: Combining conceptual CFG analysis with profiling, loop testing, and stress testing helps identify resource-intensive execution paths and improves the stability of the library management module.

Question 6: Mobile Payment Gateway — State Transition Testing

1. Problem Analysis

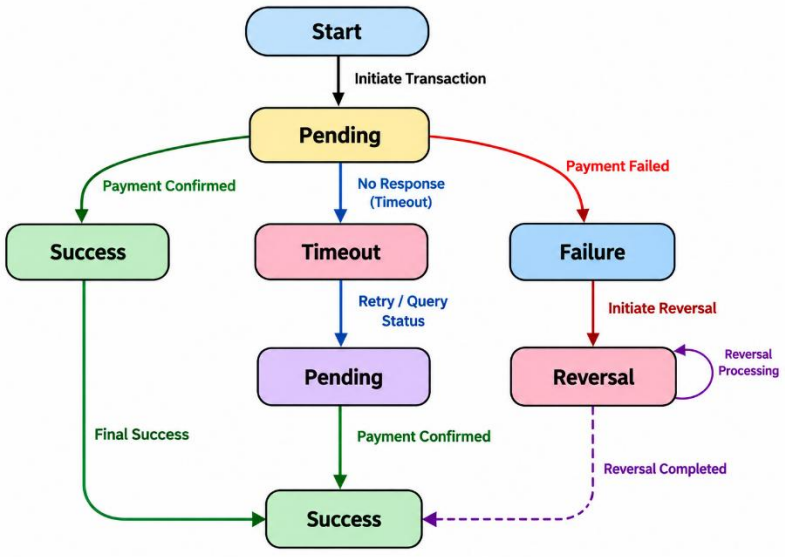
The payment gateway returns JSON responses representing Pending, Success, Reversal, and Timeout states. These states describe the transaction lifecycle. Reverse engineering requires identifying legal transitions, their triggers, and the conditions that make a transition valid.

The actual JSON responses are not included in the assessment, so the following state machine is an illustrative model.

2. State Decomposition

- Pending: The transaction has started but is not yet confirmed.
- Success: The payment has been completed successfully.
- Reversal: A previous debit or transaction is being reversed.
- Timeout: A response was not received within the expected time.

3. State Transition Diagram



A real payment gateway must define whether a timed-out transaction can return to Pending, later become Success, or enter Reversal. These rules must be verified from the actual API contract.

4. State Transition Test Cases

Test Case	Initial State	Event	Expected Result
TC1	None	Initiate payment	Pending
TC2	Pending	Payment confirmed	Success
TC3	Pending	Response timeout	Timeout or defined pending state
TC4	Pending	Payment fails after debit	Reversal initiated if required
TC5	Timeout	Status query confirms payment	Update to confirmed state
TC6	Success	Duplicate success callback	No duplicate debit
TC7	Reversal	Reversal confirmed	Reversal completed
TC8	Success	Invalid transition to Pending	Reject or ignore according to API rules

5. Data Constraints

Tests should verify:

- Unique transaction IDs.
- Valid transaction amounts and currencies.
- Consistent transaction status values.
- Valid timestamps and response codes.
- Idempotency for repeated requests.
- Correct relationship between original transactions and reversals.

6. Improved Testing Strategy

1. Identify all states and transition triggers.
2. Document permitted and forbidden transitions.
3. Test every legal transition.
4. Test invalid transitions and repeated callbacks.
5. Verify timeout recovery and status reconciliation.
6. Check transaction IDs and amount consistency.
7. Test network failures and delayed responses.
8. Automate state transition and regression tests.

Conclusion: State transition testing verifies both the sequence of payment states and the conditions governing each transition. It reduces the risk of duplicate debits, incomplete reversals, and inconsistent transaction statuses.

Question 7: Encrypted Communication Module — Fuzzing and Boundary Value Analysis

1. Problem Analysis

The encrypted communication module accepts input parameters and produces encrypted outputs, but its source code is unavailable. Fuzzing can reveal how the module handles malformed, unexpected, or extreme inputs. BVA can systematically test values near defined input limits.

The module can be reverse engineered by observing successful responses, validation errors, crashes, and unexpected outputs.

2. Functional Decomposition

1. Input Validation: Checks data types, lengths, formats, and required fields.
2. Parameter Processing: Normalizes accepted input.
3. Encryption: Encrypts validated data using the configured algorithm.
4. Output Generation: Returns ciphertext or a response.
5. Error Handling: Rejects malformed inputs and handles failures safely.

3. Boundary Value Analysis

Assume the application accepts messages of 1 to 1,024 bytes. This is an illustrative input limit.

Test Case	Input Length	Expected Result
TC1	0 bytes	Reject empty input if prohibited
TC2	1 byte	Accept minimum valid input
TC3	2 bytes	Accept valid input
TC4	1,023 bytes	Accept valid input
TC5	1,024 bytes	Accept maximum permitted input
TC6	1,025 bytes	Reject or safely handle oversized input
TC7	Malformed encoding	Return validation error
TC8	Missing required parameter	Return validation error
TC9	Unexpected data type	Reject safely

The permitted length and empty-input rules must be confirmed from the actual interface specification.

4. Fuzzing Strategy

Fuzzing involves supplying varied, malformed, or unexpected inputs to identify failures.

Examples include:

- Empty and extremely long strings.
- Random byte sequences.
- Invalid character encodings.
- Missing parameters.
- Unexpected JSON structures.
- Incorrect data types.
- Repeated and malformed delimiters.
- Invalid cryptographic parameter lengths.

The tester should record crashes, timeouts, unexpected outputs, and validation failures.

5. Preventing Buffer Overflow and Injection Vulnerabilities

- Validate input lengths before processing.
- Use memory-safe operations and bounds-checked buffers.
- Reject malformed data before encryption.
- Use parameterized queries if inputs interact with a database.
- Avoid constructing executable commands from untrusted input.
- Apply safe parsing and strict type validation.
- Test the handling of malformed inputs using controlled fuzzing.

Encryption alone does not prevent buffer overflows or injection vulnerabilities; secure input handling is also required.

6. Improved Testing Strategy

1. Identify input fields and their permitted ranges.
2. Apply BVA to input-length limits.
3. Use ECP for valid and invalid data formats.
4. Perform fuzz testing on each input parameter.
5. Verify error handling and resource limits.
6. Check that invalid requests do not crash the service.

7. Perform regression testing for every discovered defect.

Conclusion: Combining BVA, ECP, and fuzzing improves validation coverage and helps identify input-handling vulnerabilities before deployment.

Question 8: Insurance Premium Calculator — Branch and Path Coverage

1. Problem Analysis

The insurance premium calculator has 100% statement coverage but still contains defects in multi-condition if statements. This indicates that the tests execute the statements but do not adequately test the different decision outcomes or combinations of conditions.

Statement coverage alone is insufficient for verifying complex conditional logic.

2. Example Conditional Logic

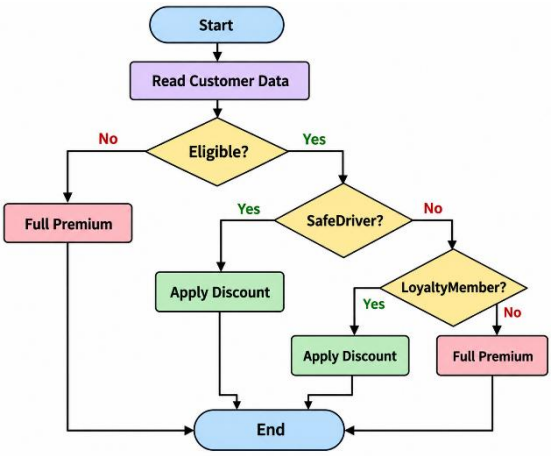
Consider this illustrative rule:

Premium Discount = Eligible AND (SafeDriver OR LoyaltyMember)

Where:

- Eligible indicates that the customer qualifies for a discount.
- SafeDriver indicates an acceptable driving record.
- LoyaltyMember indicates membership in a qualifying loyalty program.

3. Conceptual Control Flow Graph



The CFG represents a simplified conditional structure. The actual CFG must reflect the real source code.

4. Independent Execution Paths

1. Eligible = False → Full Premium.
2. Eligible = True → SafeDriver = True → Apply Discount.
3. Eligible = True → SafeDriver = False → LoyaltyMember = True → Apply Discount.
4. Eligible = True → SafeDriver = False → LoyaltyMember = False → Full Premium.

These paths cover the four main outcomes of the simplified decision structure.

5. Test Cases

Test Case	Eligible	Safe Driver	Loyalty Member	Expected Result
TC1	False	False	False	Full premium
TC2	True	True	False	Apply discount
TC3	True	False	True	Apply discount
TC4	True	False	False	Full premium
TC5	False	True	True	Full premium

The expected discount amount must be calculated using the actual insurance business rules.

6. Branch and Path Coverage

- Statement coverage ensures every executable statement runs.
- Branch coverage ensures every decision outcome executes.
- Path testing examines combinations of execution paths.

The five test cases above cover the principal branches of the simplified logic. They do not establish complete path coverage for the actual calculator without its source code.

Complete path coverage can become impractical when loops or many independent decisions are present.

7. Improved Testing Strategy

1. Construct the CFG from the actual source code.
2. Identify decision nodes and independent paths.

3. Calculate cyclomatic complexity.
4. Test all relevant Boolean combinations.
5. Verify premium calculations for each path.
6. Test invalid customer data and boundary values.
7. Add regression tests for previously discovered calculation defects.

Conclusion: Branch and basis path testing provide better assurance than statement coverage alone. They help identify errors in multi-condition decisions and reduce the risk of incorrect insurance premium calculations.

Question 9: Cloud-Based File Synchronization Tool

1. Problem Analysis

The cloud-based file synchronization tool has reported lost-file defects associated with file size, file type, and network speed. These patterns suggest that the original equivalence classes may not adequately cover large files, unusual formats, and slow or unstable network connections.

The objective is to reconstruct the input classes and integrate BVA with stress testing.

2. Functional Decomposition

1. File Selection: Identifies files to synchronize.
2. File Validation: Checks size, type, and metadata.
3. Change Detection: Identifies new or modified files.
4. Upload and Download: Transfers file data.
5. Synchronization Management: Tracks progress and resolves conflicts.
6. Error Recovery: Handles interrupted transfers.
7. Verification: Confirms file integrity after synchronization.

3. Equivalence Class Partitioning

Input Factor	Equivalence Class	Expected Behaviour
File size	Empty file	Handle according to requirements
File size	Normal file	Synchronize successfully
File size	Maximum permitted size	Synchronize successfully
File size	Above maximum	Reject safely or report an error

File type	Supported format	Synchronize
File type	Unsupported format	Apply defined rejection or handling
Network	Fast and stable	Complete transfer
Network	Slow connection	Continue or retry safely
Network	Interrupted connection	Recover without data loss

4. Boundary Value Analysis

Assume the maximum permitted file size is 100 MB. This is an illustrative limit.

Test Case	File Size	Expected Result
TC1	0 MB	Handle empty file according to requirements
TC2	1 MB	Synchronize successfully
TC3	99 MB	Synchronize successfully
TC4	100 MB	Accept maximum permitted size
TC5	101 MB	Reject or safely handle oversized file

Additional tests should examine the maximum filename length, number of files per batch, and permitted metadata sizes where relevant.

5. Stress Testing Strategy

- Synchronize many files simultaneously.
- Transfer large files under slow network conditions.
- Interrupt the connection during an upload.
- Restart the application during synchronization.
- Modify a file while synchronization is in progress.
- Test simultaneous updates from multiple devices.
- Verify retry mechanisms and conflict resolution.

6. Preventing Data Loss

The application should verify file integrity using checksums or hashes, maintain reliable synchronization metadata, and resume interrupted transfers safely. It should also prevent incomplete files from being treated as fully synchronized.

7. Improved Testing Strategy

1. Identify valid and invalid file classes.
2. Apply BVA to file-size and metadata limits.
3. Test supported and unsupported file types.
4. Perform stress testing under different network conditions.
5. Verify recovery after interruptions.
6. Compare file checksums before and after synchronization.
7. Test simultaneous modifications and conflict resolution.
8. Perform regression testing after every data-loss defect is corrected.

Conclusion: ECP identifies representative file and network conditions, BVA targets critical limits, and stress testing evaluates behaviour under heavy load. Together, they reduce the risk of lost, incomplete, or corrupted files.

Question 10: E-Commerce Search and Filter Module — Decision Table Testing

1. Problem Analysis

The e-commerce search and filter module generates SQL queries based on user-selected search parameters. The source code is unavailable, so the filtering logic must be reconstructed by observing the query parameters and resulting datasets.

The main objective is to identify how AND, OR, and NOT conditions combine and create test cases covering important search combinations.

2. Functional Decomposition

The search and filter module consists of:

1. Search Input: Accepts keywords and search terms.
2. Category Filter: Restricts results to selected categories.
3. Price Filter: Selects products within a specified price range.
4. Availability Filter: Displays available or unavailable products.
5. Query Builder: Combines selected conditions using AND, OR, and NOT.
6. Database Retrieval: Executes the generated SQL query.
7. Result Display: Shows products matching the search conditions.

3. Decision-Making Logic

Consider the following illustrative conditions:

- C1: Product matches the search keyword.
- C2: Product belongs to the selected category.
- C3: Product price is within the selected range.
- C4: Product is available.

For example:

Keyword AND Category AND Price AND Availability

All four conditions must be satisfied for a product to appear.

For an OR-based search:

Keyword AND (Category OR Availability)

A product must match the keyword and satisfy at least one of the other conditions.

For a NOT-based search:

Keyword AND NOT(Category)

Products matching the keyword are returned, excluding those belonging to the specified category.

4. Decision Table

The following table illustrates the AND-based filtering rule. Y means True and N means False.

Test Case	Keyword Match	Category Match	Price Match	Available	Expected Result
TC1	Y	Y	Y	Y	Include product
TC2	N	Y	Y	Y	Exclude product
TC3	Y	N	Y	Y	Exclude product
TC4	Y	Y	N	Y	Exclude product
TC5	Y	Y	Y	N	Exclude product
TC6	N	N	Y	Y	Exclude product
TC7	Y	Y	Y	Y	Include product

TC1 and TC7 represent the same logical combination and can be merged into one test unless they represent different input scenarios.

5. Test Cases for AND, OR, and NOT

Test Case	Filter Expression	Expected Result
TC1	Keyword AND Category	Return products satisfying both conditions
TC2	Keyword OR Category	Return products satisfying either condition
TC3	Keyword AND NOT Category	Exclude products in the specified category
TC4	Category AND Price	Return products matching both filters
TC5	Price OR Availability	Return products satisfying at least one condition
TC6	No filters selected	Display all permitted products or follow the defined default
TC7	Invalid price range	Display a validation error
TC8	No matching products	Display an appropriate empty-result message

6. SQL Query Validation

For a price and category filter, a parameterized query could be:

```
SELECT *
FROM Products
WHERE category = ?
  AND price BETWEEN ? AND ?;
```

The placeholders represent parameters supplied separately from the SQL statement. This helps prevent SQL injection.

Tests should verify that:

- AND conditions restrict results correctly.
- OR conditions include all qualifying results.
- NOT conditions exclude the intended records.
- Parentheses preserve the intended condition precedence.
- Invalid inputs do not cause query errors.
- User input cannot alter the query structure.

7. Completeness Evaluation

A decision table helps systematically identify important combinations of filter conditions. However, four independent Boolean conditions produce \(2^4 = 16\) possible combinations.

For larger numbers of filters, the number of combinations increases rapidly. Therefore, the testing strategy should combine decision table testing with equivalence partitioning, boundary testing for price ranges, and additional tests for high-risk combinations.

8. Improved Testing Strategy

1. Identify all search parameters and filtering rules.
2. Reconstruct the AND, OR, and NOT conditions.
3. Create a decision table for the relevant combinations.
4. Verify returned products against expected results.
5. Test minimum and maximum price boundaries.
6. Test empty results, invalid inputs, and missing parameters.
7. Verify SQL injection protection and parameterized queries.
8. Automate regression tests for previously identified defects.

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation